```
$ gcc revisao.c -o prova && ./prova --amanha
```

Linguagem de Programação

Guia de estudo para a prova · Fatec · ADS · 2º semestre

Este guia resume as 12 aulas e as 4 listas de exercícios da disciplina, na mesma ordem dos slides do professor. Cada conceito traz o que ele é, um exemplo de código de como usar, e três respostas rápidas: por que usar, quando usar e quando não usar. Os quadros "cai na prova" trazem as respostas comentadas dos exercícios de rastreamento ("o que será impresso?") que aparecem nas aulas e nas listas. Leia na ordem e, no fim, revise o checklist de pegadinhas.

01 Revisão
if/else, laços, vetores e matrizes · Aula 1

02 Funções
definição, protótipo, escopo, vetores como parâmetro · Aula 2

03 Structs
registros, vetores de struct, typedef · Aula 3

04 Ordenação e busca
selection, bubble, insertion, sequencial, binária · Aula 4

05 Ponteiros
& e *, referência, malloc/free, TAD · Aulas 5–8

06 Arquivos
texto, binário, fseek, argc/argv · Aulas 9–10

07 Recursão
indução, pilha, Hanoi, mergesort, backtracking · Aulas 11–12

08 Checklist
pegadinhas e respostas das listas · Listas 5–8

01 Revisão

Condicionais, laços de repetição, vetores e matrizes.

`// 01 · revisao`

if / else aninhado

O que é. O else sempre pertence ao if mais próximo que ainda não tem else — a indentação não muda isso. Use chaves para deixar claro o dono de cada bloco. Em C, condição ≠ 0 é verdadeira.

```c
if (cond1) {
  if (cond2)
    comando1;
} else             // else do cond1
  comando2;        // roda se cond1 for falsa

if (a = 3)         // ERRO: atribuição, sempre V
  printf("sempre entra");
if (a == 3)        // certo: comparação
  printf("a vale 3");
```

`por_que_usar()`	Escolher caminhos diferentes do programa conforme o resultado de um teste.
`quando_usar()`	Quando o próximo passo depende de um teste: faixas de valores, par/ímpar, menus.
`quando_NAO_usar()`	Sem chaves em ifs aninhados, e nunca = no lugar de == dentro da condição.

`// 01 · revisão`

while × do-while × for

Os três repetem comandos enquanto a condição é verdadeira; a diferença é quando ela é testada.

Laço	Testa a condição	Executa no mínimo	Use quando
while	antes de cada volta	0 vezes	não se sabe quantas repetições (ex.: até digitar 0)
do-while	depois de cada volta	1 vez	o bloco precisa rodar ao menos uma vez (menu, validar entrada)
for	antes (início; condição; passo)	0 vezes	a quantidade de repetições é conhecida (percorrer vetor)

for (início; cond; passo) { cmd; } ≡ início; while (cond) { cmd; passo; }

Condição falsa logo de cara → nunca entra no laço. Condição sempre verdadeira → laço infinito.

`// 01 · revisão`

Vetores e matrizes

O que é. Vetor: coleção de variáveis do mesmo tipo com um só nome, em posições contíguas da memória. Índices vão de 0 a tam−1. Matriz: tipo nome[linhas][colunas], com linhas × colunas variáveis.

```c
double v1[5], v2[5], res = 0.0;
for (i = 0; i < 5; i++)
  scanf("%lf", &v1[i]);
// ... lê v2 do mesmo jeito
for (i = 0; i < 5; i++)
  res = res + v1[i] * v2[i];  // produto interno

int mat[4][4];                // 16 inteiros
mat[i][j] = 0;                // linha i, coluna j
```

`por_que_usar()`	Guardar e processar muitos dados do mesmo tipo com um laço, sem criar 100 variáveis.
`quando_usar()`	Listas de notas, leituras, tabelas (matriz), quando o tamanho máximo é conhecido.
`quando_NAO_usar()`	Acessar índice fora de 0..tam−1 (comportamento anômalo) ou juntar tipos diferentes (→ struct).

`// 01 · cai na prova`

Cai na prova: O que será impresso?

if aninhado com a = 5, −12 e 9

```c
if (a > 3) {
  if (a < 7) printf("a");
} else {
  if (a > -10) printf("b");
  else printf("c");
}
```

saída →

a = 5 → a · a = −12 → c · a = 9 → nada (o if interno não tem else)

do-while com o usuário digitando 0

```c
int i = 1, n;
scanf("%d", &n);       // digita 0
do {
  printf("%d ", i);
  i++;
} while (i <= n);
```

saída →

Imprime 1 — o do-while executa antes de testar. Com while, não imprimiria nada.

02 Funções

Modularizar: quebrar um problema grande em partes menores.

// 02 · funções

Funções: definir e chamar

O que é. Bloco de comandos com nome, executado quando é chamado. Tem tipo de retorno, parâmetros e return, que devolve o valor e encerra a função na hora. Depois, o fluxo volta para onde ela foi chamada.

```c
int soma(int a, int b) {     // tipo nome(params)
  int c = a + b;
  return c;                  // devolve e PARA
  printf("nunca roda");
}
int main() {
  int r = soma(12, 90);      // r = 102
  printf("%d\n", soma(r, 1)); // chamada é expressão
  return 0;                  // 0 = deu tudo certo
}
```

por_que_usar()	Modularização: código menor e legível, reaproveitado e sem trechos repetidos.
quando_usar()	Quando um trecho se repete ou tem tarefa clara: ler número, testar primo, potência.
quando_NAO_usar()	Para devolver um vetor (int[] f() não compila) ou vários valores (→ ponteiros).

// 02 · funções

void, main e protótipos

O que é. void: a função não devolve nada (sem return de valor). main é chamada pelo sistema operacional e retorna int (0 = sucesso). Protótipo: a 1ª linha da função com ; — declara antes de usar.

```c
#include <stdio.h>
float soma(float a, float b);  // protótipos
void imprime(int n);           // no topo

int main() {
  imprime(10);
  printf("%f\n", soma(2, 3));
  return 0;
}
float soma(float a, float b) { return a + b; }
void imprime(int n) { printf("Numero %d\n", n); }
```

por_que_usar()	Organiza o arquivo: includes, protótipos, globais, main e depois as funções.
quando_usar()	Sempre que a função for definida depois do ponto em que é chamada (ex.: abaixo da main).
quando_NAO_usar()	Chamar função definida depois da main sem protótipo: pode dar erro de compilação.

// 02 · funções

Passagem por valor × vetores

O que é. Variável simples é COPIADA para o parâmetro: mudar dentro da função não muda fora. Vetor não é copiado — a função recebe o endereço e altera o original.

```
int incr(int x) { x = x + 1; return x; }
void zera(int v[], int tam) {
  for (int i = 0; i < tam; i++) v[i] = 0;
}
int a = 2, b = incr(a);   // a = 2, b = 3
int x[5] = {1, 2, 3, 4, 5};
zera(x, 5);               // x = {0,0,0,0,0}

void f(int m[][10], int n); // ok: omite só a 1ª
void g(int m[10][], int n); // ERRO
```

`por_que_usar()`	Saber se a função mexe no dado original evita bugs e resolve as questões de rastreio.
`quando_usar()`	Padrão para vetores: int v[], int tam — a função serve para vetores de qualquer tamanho.
`quando_NAO_usar()`	Esperar que int/float/char passado por valor mude fora da função (→ ponteiros).

// 02 · funções

Escopo: locais e globais

O que é. Local: declarada dentro da função (parâmetros também) e deixa de existir quando ela termina. Global: fora de qualquer função, visível em todas. Uma local de mesmo nome esconde a global.

```
int x;                      // global
void fun1() { x = x + 1; printf("fun1: %d\n", x); }
void fun2() { int x = 3; printf("fun2: %d\n", x); }
int main() {
  x = 1;
  fun1();                   // fun1: 2 (global)
  fun2();                   // fun2: 3 (local)
  printf("main: %d\n", x);  // main: 2
}
```

`por_que_usar()`	Entender quem enxerga o quê é a base das questões “o que será impresso?”.
`quando_usar()`	Variáveis locais por padrão; parâmetros para passar dados entre funções.
`quando_NAO_usar()`	Evite globais: qualquer função altera, cria dependência e piora a leitura do código.

// 02 · cai na prova

Cai na prova: O que será impresso?

Exemplo 2 — local na main

```
int x = 1;                  // global
int main() {
  int x = 1;                // local da main!
  fun1(); fun2();
  printf("main: %d\n", x);
}
void fun1() { x = x + 1; } // mexe na global
void fun2() { int x = 4; } // imprime a sua
```

saída →

fun1: 2 · fun2: 4 · main: 1 — a main imprime a SUA x local, que ninguém alterou.

Exemplo 3 — parâmetro com nome x

```c
int x = 1;
int main() {
  x = 2;
  fun1(); fun2(x);
  printf("main: %d\n", x);
}
void fun1() { x = x + 1; }      // global
void fun2(int x) { x = x + 1; } // cópia
```

saída →

fun1: 3 · fun2: 4 · main: 3 — fun2 alterou só a cópia (parâmetro é local).

// 02 · cai na prova

Cai na prova: Funções chamando funções

fun1 chama fun2 — valor de c?

```c
int fun1(int a) {
  a = a + 1;
  return fun2(a);
}
int fun2(int b) { return 2 * b; }
int c = 5;
c = fun1(c);    // ?
```

saída →

c = 12 → fun1: 5 + 1 = 6; fun2: 2 × 6 = 12. Qualquer função chama outra (menos a main).

Lista 5 · questão 1 (D = último dígito do RA)

```c
int j = 1;                  // global
int main() { int i, j;      // j local
  a = D % 2 == 0 ? 2 : 3;
  printf("%d\n", fun1(2, 4));
  // i, j de 1 a 2: fun1(a, i + j)
}
int fun1(int a, int b)      // a^b + j global
```

saída →

Globais: j. Locais: main → i, j, a; fun1 → a, b, i, p. D par: 17 5 9 9 17 · D ímpar: 17 10 28 28 82

03 Structs

Registros: várias variáveis, até de tipos diferentes, juntas.

// 03 · structs

struct: agrupando dados

O que é. Registro agrupa variáveis que fazem sentido juntas (nome + nota). Cada campo é acessado com ponto, variavel.campo, e se comporta como uma variável normal do seu tipo.

```c
struct Aluno {              // novo tipo
  char nome[80];
  float nota;
};                          // declare fora das funções
struct Aluno a, b;
strcpy(a.nome, "Helen");    // string: strcpy
a.nota = 8.6;
fgets(b.nome, 80, stdin);              // campo a campo
b.nome[strlen(b.nome) - 1] = '\0';     // tira o \n
b = a;                      // copia TODOS os campos
```

`por_que_usar()`	Representar uma entidade real (aluno, paciente, produto) como uma unidade só.
`quando_usar()`	Quando vários dados diferentes descrevem a mesma coisa; declarado fora, todas as funções usam.
`quando_NAO_usar()`	Ler ou imprimir a struct inteira de uma vez: scanf e printf são sempre campo a campo.

`// 03 · structs`

Vetor de struct, funções e typedef

O que é. struct se comporta como tipo básico: pode ser parâmetro (é copiada) e retorno de função. typedef cria um apelido para um tipo: typedef float nota; ou typedef struct Aluno Aluno;

```c
typedef struct Aluno Aluno;   // agora é só Aluno
Aluno leAluno() {             // retorna uma struct
  Aluno aux;
  scanf("%f", &aux.nota);
  return aux;
}
void listar(Aluno t[], int n) {
  for (int i = 0; i < n; i++)
    printf("%.2f\n", t[i].nota); // t[i].campo
}
Aluno turma[60];
turma[0] = leAluno();
```

`por_que_usar()`	Cadastros inteiros (turma, estoque) tratados com laços e funções reaproveitáveis.
`quando_usar()`	typedef para nomes curtos e claros; vetor de struct para listas de registros.
`quando_NAO_usar()`	Passar struct grande por valor dentro de laço: copia tudo (→ ponteiro e p→campo).

04 Ordenação e busca

Deixar dados em ordem e encontrar uma chave.

`// 04 · ordenação`

Selection Sort

O que é. A cada passo i, acha o MENOR elemento de i até o fim e troca com a posição i. (5,3,2,1,90,6) → (1,3,2,5,90,6) → (1,2,3,5,90,6) → … → (1,2,3,5,6,90).

```c
void selectionSort(int v[], int tam) {
  int i, j, min;
  for (i = 0; i < tam - 1; i++) { // o último sobra
    min = i;
    for (j = i + 1; j < tam; j++)
      if (v[min] > v[j]) min = j; // acha o menor
    troca(&v[i], &v[min]);
  }
}
void troca(int *a, int *b) {
  int aux = *a;  *a = *b;  *b = aux;
}
```

`por_que_usar()`	Ideia simples e poucas trocas: no máximo tam − 1.
`quando_usar()`	Vetores pequenos; quando trocar elementos de lugar é caro.
`quando_NAO_usar()`	Vetores grandes: percorre o resto inteiro mesmo se já estiver ordenado.

// 04 · ordenação

Bubble Sort

O que é. Compara vizinhos v[j] e v[j+1] e troca se estiverem fora de ordem. Ao fim de cada passada, o MAIOR "borbulha" para o final — por isso a passada seguinte vai uma posição a menos.

```c
void bubbleSort(int v[], int tam) {
  int i, j;
  for (i = tam - 1; i > 0; i--)
    for (j = 0; j < i; j++)      // trocas até i
      if (v[j] > v[j + 1])
        troca(&v[j], &v[j + 1]);
}
// 1ª passada: (5, 3, 2, 1, 90, 6)
//          → (3, 2, 1, 5, 6, 90)  90 no lugar
```

por_que_usar()	O mais fácil de lembrar e escrever: só compara vizinhos.
quando_usar()	Vetores pequenos ou quase ordenados; questões de rastreio passo a passo.
quando_NAO_usar()	Muitos dados: faz muitas comparações e trocas.

// 04 · ordenação

Insertion Sort

O que é. Como arrumar cartas na mão: de 0 a i−1 já está ordenado; guarda v[i] em aux, empurra os maiores uma casa para frente e encaixa aux em j+1.

```c
void insertionSort(int v[], int tam) {
  int i, j, aux;
  for (i = 1; i < tam; i++) {
    aux = v[i];
    j = i - 1;
    while (j >= 0 && v[j] > aux) {
      v[j + 1] = v[j];     // empurra para frente
      j--;
    }
    v[j + 1] = aux;        // encaixa no lugar
  }
}
```

por_que_usar()	Rápido quando o vetor já está quase ordenado.
quando_usar()	Inserir itens numa lista que já está em ordem; vetores pequenos.
quando_NAO_usar()	Vetores grandes e bagunçados: muitos deslocamentos.

// 04 · ordenação

Comparando os três

Algoritmo	Ideia	Laço externo	Garantia após o passo i
Selection	seleciona o menor e troca	i = 0 até tam − 2	posições 0..i já definitivas
Bubble	troca vizinhos fora de ordem	i = tam − 1 até 1	o maior de 0..i foi para a posição i
Insertion	insere v[i] no trecho ordenado	i = 1 até tam − 1	0..i ordenado (ainda pode mudar)

Ordem decrescente? Inverta a comparação: v[min] < v[j] · v[j] < v[j+1] · v[j] < aux.

Selection vai só até tam − 2: quando sobra um elemento, ele já é o maior e está no lugar.

// 04 · busca

Busca sequencial × binária

O que é. Ambas devolvem o índice da chave ou −1. A sequencial testa um por um; a binária olha o meio e descarta metade a cada passo — só funciona com o vetor ORDENADO.

```
int buscaSeq(int v[], int tam, int chave) {
  for (int i = 0; i < tam; i++)
    if (v[i] == chave) return i;
  return -1; }
int buscaBin(int v[], int tam, int chave) {
  int ini = 0, fim = tam - 1, meio;
  while (ini <= fim) {        // pelo menos 1 elemento
    meio = (ini + fim) / 2;
    if (v[meio] == chave) return meio;
    if (v[meio] > chave) fim = meio - 1;
    else ini = meio + 1;
  }
  return -1; }
```

`por_que_usar()`	Encontrar um registro por chave única: RA, RG, CPF, nº da carteira.
`quando_usar()`	Sequencial: vetor desordenado ou pequeno. Binária: ordenado e com muitas buscas.
`quando_NAO_usar()`	Binária em vetor desordenado, ou com inserções/remoções constantes (manter ordem custa).

// 04 · busca

Eficiência: quantos acessos ao vetor?

Caso	Busca sequencial	Busca binária
Melhor caso	1 acesso (chave em v[0])	1 acesso (chave no meio)
Pior caso	tam acessos (última ou ausente)	$\log_2$ tam acessos
Caso médio	(tam + 1) / 2	≈ $\log_2$ tam − 1
1 milhão de itens	≈ 500.000 acessos	≈ 20 acessos

A cada passo a binária divide o vetor por 2: tam / 2^x = 1 → x = $\log_2$ tam.

Eficiência = recursos usados (tempo, memória). Aqui contamos acessos a posições do vetor.

05 Ponteiros

Variáveis que guardam endereços de memória.

// 05 · ponteiros

Ponteiros: & e *

O que é. Ponteiro é uma variável que guarda o ENDEREÇO de outra variável do tipo indicado. & devolve o ende-

reço de uma variável; * acessa o conteúdo do endereço apontado.

```c
int a = 90;
int *p;            // p guarda endereço de int
p = &a;            // p aponta para a
*p = 11;           // muda a: agora a == 11
printf("%d", *p);  // 11

int *q = NULL;     // não aponta para nada
if (q != NULL) printf("%d", *q);
int *c;  *c = 13;  // ERRO: endereço inválido
x = b * (*c);      // use ( ) ao multiplicar
```

`por_que_usar()`	Alterar variáveis de outra função, trabalhar com vetores e alocar memória.
`quando_usar()`	Devolver vários valores, passar vetores e structs, alocação dinâmica.
`quando_NAO_usar()`	Sem inicializar (aponte para algo ou NULL) ou com tipo trocado: int* em double dá lixo.

`// 05 · ponteiros`

Passagem por referência

O que é. Em C só existe passagem por valor. Para a função alterar a variável de quem chamou, passe o ENDEREÇO (&x) e use *p dentro dela — o efeito é o de uma passagem por referência.

```c
void troca(int *a, int *b) {
  int aux = *a;  *a = *b;  *b = aux;
}
void maxMin(int v[], int tam, int *min, int *max) {
  *min = v[0];  *max = v[0];
  for (int i = 0; i < tam; i++) {
    if (v[i] < *min) *min = v[i];
    if (v[i] > *max) *max = v[i];
  }
}
troca(&x, &y);            // x e y trocam
maxMin(v, 9, &mn, &mx);   // dois “retornos”
```

`por_que_usar()`	Uma função só retorna um valor; com ponteiros ela devolve quantos precisar.
`quando_usar()`	Trocar valores, achar min e max juntos, ler dados (o scanf usa & por isso).
`quando_NAO_usar()`	Esquecer o & na chamada (vai o valor, não o endereço) ou mexer em p em vez de *p.

`// 05 · ponteiros`

Ponteiros e vetores

O que é. O nome do vetor guarda o endereço da 1ª posição; por isso um ponteiro pode apontar para ele e ser usado como vetor (p[i]). Diferença: o vetor tem endereço FIXO, o ponteiro não.

```c
int a[] = {1, 2, 3, 4, 5};
int *p = a;          // ok: p aponta para a[0]
p[2] = 30;           // altera a[2]!
int b[5];
b = a;               // ERRO de compilação

int *b2 = a;
for (i = 0; i < 5; i++) b2[i] = i * i;
// a agora vale {0, 1, 4, 9, 16}
```

`por_que_usar()`	Explica por que vetores mudam dentro de funções: vai o endereço, não uma cópia.
`quando_usar()`	Percorrer ou alterar vetores por ponteiro e guardar vetores alocados dinamicamente.
`quando_NAO_usar()`	Copiar vetor com b = a: não compila (vetor) ou só aponta para o mesmo lugar (ponteiro).

// 05 · cai na prova

Cai na prova: Rastreando ponteiros

Exercício da aula 5

```c
int a = 3, b = 2, *p = NULL, *q = NULL;
p = &a;
q = p;
*q = *q + 1;
q = &b;
b = b + 1;
printf("%d\n%d\n", *q, *p);
```

saída →

3 e 4 → *q aponta para b (2 + 1 = 3); *p aponta para a, que virou 4.

Lista 6 · questão 1 (&x = 1000, &y = 1004)

```c
int x = 10, y = 20;
int *p1, *p2;
p1 = &x;
p2 = &y;
(*p1)++;
```

saída →

x = 11 · y = 20 · &x = 1000 · &y = 1004 · p1 = 1000 · p2 = 1004 · *p1 + *p2 = 31 · *(&x) = 11 · &(*p2) = 1004

// 05 · cai na prova

Cai na prova: Ponteiros em funções e structs

Lista 6 · questão 3

```c
void f1(int v[], int n, int *a) {
  *a = v[0];
  for (i = 1; i < n; i++)
    if (v[i] % 2 == 0 && *a < v[i])
      *a = v[i];
}
int v[] = {14, 7, 8, 51, 13, 20, 2};
int h = 15;  f1(v, 7, &h);
```

saída →

h = 20 → *a começa em 14 e vira o maior par do vetor; o 15 inicial é sobrescrito.

Lista 7 · questão 6

```c
T a, b, *c, *d;  // T = struct { x, y }
c = &a;  a.x = 2;  a.y = 4;
b.x = 2;  b.y = 2;
d = c;
f1(d);   // f2(&(a->x)); f2(&(a->y));
b = *d;  // f2 faz *b = 2 * (*b)
```

saída →

x: 4 --- y: 8 → d aponta para a; f2 dobra cada campo; b = *d copia a inteira.

// 05 · ponteiros

malloc, calloc e free

O que é. Alocação dinâmica (stdlib.h): pede memória durante a execução, do tamanho necessário. Devolve o endereço do bloco ou NULL se falhar. Toda memória alocada deve ser liberada com free.

```c
#include <stdlib.h>
int n;  scanf("%d", &n);
double *v = malloc(n * sizeof(double));
int *z = calloc(n, sizeof(int));   // já zera
if (v == NULL) { /* faltou memória */ }
for (i = 0; i < n; i++) v[i] = 2 * i;
free(v);  free(z);                 // devolve

char *concatena(char *s1, char *s2);
// retorna malloc; quem chama faz o free
```

`por_que_usar()`	Tamanho decidido em execução: sem #define MAX e sem recompilar o programa.
`quando_usar()`	Quantidade de dados desconhecida. malloc é mais rápido; calloc entrega tudo zerado.
`quando_NAO_usar()`	Dar free antes de terminar de usar (ex.: dentro da função que retorna o ponteiro) ou esquecer o free.

`// 05 · ponteiros`

Memória do programa e erros comuns

Os quatro segmentos. Código executável: o binário do programa. **Dados estáticos**: variáveis globais e static, que existem durante toda a execução. **Heap**: tudo que é criado com malloc/calloc; o limite é a memória do computador. **Pilha**: variáveis locais e chamadas de função, criadas na chamada e removidas no fim dela; o sistema operacional limita o seu tamanho.

`erro: double v[n] com n enorme`	Em C99 dá para declarar vetor com tamanho lido em execução, mas ele vai para a PILHA. Com n = 2.000.000 o programa morre com segmentation fault. Com malloc o vetor vai para o heap e funciona.
`erro: usar depois do free`	v2 = v1; free(v1); v2[i] = 0; — v2 aponta para a mesma região, que já foi liberada. O resultado muda conforme outras alocações (o exemplo do calloc em v3 mostra isso).
`erro: free dentro da função que retorna`	Em concatena, dar free(sres) antes do return devolve memória já liberada. Quem chama a função é que deve dar free(s3) depois de usar.

`// 05 · ponteiros`

Vetor dinâmico: cresce e encolhe

O que é. size = quantos elementos há; maxSize = espaço alocado (mínimo 4). Cheio → aloca o DOBRO e copia. Menos de 1/4 ocupado → aloca a METADE. A função sempre devolve o endereço atual.

```
int *addVet(int *v, int *size, int *maxSize, int e) {
  if (*size < *maxSize) {        // tem espaço
    v[*size] = e;  (*size)++;  return v;
  }
  int *aux = malloc(2 * (*maxSize) * sizeof(int));
  for (int i = 0; i < *size; i++) aux[i] = v[i];
  aux[*size] = e;  (*size)++;
  *maxSize = 2 * (*maxSize);
  free(v);                       // libera o antigo
  return aux;
}
vet = addVet(vet, &size, &maxSize, 7);
```

`por_que_usar()`	Usa só a memória necessária, sem limite fixo de tamanho.
`quando_usar()`	Listas que mudam de tamanho: cadastros com inclusões e remoções.
`quando_NAO_usar()`	Ignorar o retorno: escreva vet = addVet(...), pois o endereço pode ter mudado.

`// 05 · ponteiros`

Ponteiro para ponteiro e matriz dinâmica

O que é. int **p guarda o endereço de um int*. Matriz dinâmica: um vetor de ponteiros (as linhas) e cada posi-

ção recebe um vetor (as colunas). Libere cada linha e depois o vetor de linhas.

```c
int a = 5, *b = &a, **c = &b;
printf("%d %d %d", a, *b, **c);  // 5 5 5

int **mat = malloc(n * sizeof(int *));
for (i = 0; i < n; i++)
  mat[i] = malloc(m * sizeof(int));
mat[i][j] = i * j;               // usa normal
for (i = 0; i < n; i++)
  free(mat[i]);                  // 1º as linhas
free(mat);                       // depois o vetor
```

`por_que_usar()`	Matrizes com linhas × colunas definidas pelo usuário, sem desperdício.
`quando_usar()`	Cálculos com matrizes de tamanho variável (ciência e engenharia).
`quando_NAO_usar()`	Dar só free(mat): as linhas vazam. Mais eficiente: um vetor n × m com índice linearizado.

`// 05 · ponteiros`

TAD e tipo opaco: o Ponto

O que é. Tipo Abstrato de Dados: os campos ficam escondidos no .c; quem usa vê no .h só o nome do tipo (tipo opaco) e as funções. Campo via ponteiro: p→x, o mesmo que (*p).x.

```c
// Ponto.h — o que o usuário vê
typedef struct ponto Ponto;
Ponto *pto_cria(float x, float y);
void pto_libera(Ponto *p);
float pto_distancia(Ponto *p1, Ponto *p2);
// Ponto.c — escondido
struct ponto { float x; float y; };
Ponto *pto_cria(float x, float y) {
  Ponto *p = malloc(sizeof(Ponto));
  if (p != NULL) { p->x = x; p->y = y; }
  return p;
}
// main.c:  Ponto r;  q->x = 2;  → ERRO
```

`por_que_usar()`	Encapsulamento: muda-se a implementação sem quebrar quem usa a biblioteca.
`quando_usar()`	Bibliotecas reutilizáveis (ponto, número complexo, data) separadas em .h e .c.
`quando_NAO_usar()`	Acessar campos direto no main ou declarar Ponto r; — só ponteiros: Ponto *p.

06 Arquivos

Guardar dados na memória secundária, depois que o programa termina.

`// 06 · arquivos`

Por que arquivos e quais tipos

Por que arquivos. Todo programa roda na RAM (memória principal), que é rápida, mas cara e volátil: quando o programa termina ou acaba a energia, tudo se perde. A memória secundária (HD, SSD) é mais lenta, mais ba-

rata, tem mais capacidade e não depende de energia. Para guardar dados de forma persistente, o programa escreve em arquivos, identificados por nome e extensão (.txt, .c, .pdf, .html, .exe).

Dois tipos. Texto: caracteres que aparecem na tela e podem ser editados em qualquer editor (código C, .txt, HTML). **Binário**: sequência de bits no formato do programa que o gerou, não legível direto (executáveis, compactados, documentos do Word).

Caminhos. Absoluto: desde a raiz, como /home/usr1/arq.txt. **Relativo**: a partir do diretório atual, como arq.txt ou mc102/lab.c. Diretórios (pastas) contêm arquivos e outros diretórios.

// 06 · arquivos

Arquivo texto: fopen, fscanf, fprintf

O que é. FILE* aponta para o arquivo. fopen abre (NULL se falhar); fscanf lê até devolver EOF; fprintf escreve; fclose fecha e garante a gravação; rewind volta ao início.

```c
FILE *in = fopen("teste.txt", "r");
FILE *out = fopen("saida.txt", "w");
if (in == NULL) { printf("Erro"); return 1; }
char c;
while (fscanf(in, "%c", &c) != EOF) // até o fim
  fprintf(out, "%c", c);            // copia
rewind(in);                         // volta
fscanf(in, "%d", &i);  // lê int, double...
fclose(in);  fclose(out);
```

`por_que_usar()`	Na RAM tudo some quando o programa acaba; em arquivo os dados ficam guardados.
`quando_usar()`	Ler entradas grandes, salvar resultados, gerar relatórios legíveis em editor.
`quando_NAO_usar()`	Esquecer de testar NULL ou de dar fclose — os dados podem não ser gravados.

// 06 · arquivos

Modos do fopen

Modo	Faz	Se NÃO existe	Se já existe
r · rb	leitura	devolve NULL	lê desde o início
r+ · r+b	leitura e escrita	devolve NULL	mantém o conteúdo
w · wb	escrita	cria	APAGA e recria
w+ · w+b	escrita e leitura	cria	APAGA e recria
a	escrita no final (append)	cria	escreve depois do fim

O b indica arquivo binário. O indicador de posição começa no início (no fim com a) e avança sozinho a cada leitura/escrita.

remove("nome.txt") apaga um arquivo.

// 06 · arquivos

Parâmetros do programa: argc e argv

O que é. A main pode receber a linha de comando: argc conta os argumentos; argv é um vetor de strings com

cada um. argv[0] é o nome do programa.

```c
int main(int argc, char *argv[]) {
  if (argc != 2) {
    printf("Uso: ./mostra arquivo.txt\n");
    return 1;
  }
  FILE *fp = fopen(argv[1], "r");
  if (fp == NULL) return 1;
  char c;
  while (fscanf(fp, "%c", &c) != EOF)
    printf("%c", c);
  fclose(fp);
}   // $ ./mostra notas.txt  → argc = 2
```

`por_que_usar()`	Rodar o programa com entradas diferentes sem digitar nada durante a execução.
`quando_usar()`	Nome do arquivo de entrada, números de configuração (converta com atoi).
`quando_NAO_usar()`	Acessar argv[1] sem conferir argc: se o argumento não veio, o programa quebra.

`// 06 · arquivos`

Arquivo binário: fwrite, fread, fseek

O que é. Grava os bytes como estão na memória: cada item tem tamanho fixo, o arquivo fica menor e structs inteiras vão de uma vez. fseek pula direto para qualquer posição.

```c
FILE *f = fopen("alunos.bin", "w+b");
fwrite(vet, sizeof(Aluno), 4, f);    // 4 registros
fclose(f);
f = fopen("alunos.bin", "r+b");
Aluno aux;
while (fread(&aux, sizeof(Aluno), 1, f) != 0)
  if (aux.RA == 2) break;            // achou
fseek(f, -1 * sizeof(Aluno), SEEK_CUR); // volta 1
strcpy(aux.nome, "Isabela");
fwrite(&aux, sizeof(Aluno), 1, f);   // sobrescreve
fclose(f);
```

`por_que_usar()`	Acesso direto ao registro n: fseek(f, n * sizeof(T), SEEK_SET) — um mini banco de dados.
`quando_usar()`	Cadastros de structs, vetores de números, dados que só o programa vai ler.
`quando_NAO_usar()`	Quando uma pessoa precisa ler ou editar o arquivo num editor (→ arquivo texto).

`// 06 · arquivos`

fwrite, fread e fseek em detalhe

Função	Parâmetros	Devolve
fwrite	(endereço dos dados, bytes de 1 item, nº de itens, FILE*)	nº de itens gravados
fread	(endereço de memória já alocada, bytes de 1 item, nº de itens, FILE*)	nº de itens lidos — 0 no fim
fseek	(FILE*, nº de bytes, origem: SEEK_SET · SEEK_CUR · SEEK_END)	move o indicador de posição

SEEK_SET = a partir do início · SEEK_CUR = da posição atual · SEEK_END = a partir do fim.

Ler sem saber quantos itens há: while (fread(&x, sizeof(x), 1, f) != 0) { ... }

07 Recursão

Resolver o problema usando o mesmo problema, só que menor.

// 07 · recursão

Indução: a base da recursão

O que é. Para provar algo para todo n: (1) passo base, vale para n = 1; (2) hipótese, vale para n − 1; (3) passo, usando n − 1 prova-se para n. A função recursiva segue o mesmo molde.

```
// Teorema: S(n) = 1 + 2 + ... + n = n(n+1)/2
// Base:    S(1) = 1(1+1)/2 = 1          ok
// Hipótese: S(n-1) = (n-1)n/2
// Passo:    S(n) = S(n-1) + n
//                = (n-1)n/2 + 2n/2
//                = n(n+1)/2                ok
int S(int n) {
  if (n == 1) return 1;      // caso base
  return S(n - 1) + n;       // caso geral
}
```

`por_que_usar()`	Transforma um problema grande no mesmo problema menor: código curto e com garantia.
`quando_usar()`	Problemas recursivos por natureza: fatorial, Fibonacci, potência, Hanoi, mergesort.
`quando_NAO_usar()`	Sem caso base ou sem diminuir n a cada chamada: recursão infinita, a pilha estoura.

// 07 · recursão

Fatorial e a pilha de execução

O que é. n! = 1 se n ≤ 1; senão n! = n · (n − 1)!. A própria definição já é recursiva. Cada chamada de função empilha suas variáveis locais no topo da pilha; quando termina, elas são removidas. Em chamadas recursivas é como se cada chamada fosse uma função diferente, com seus próprios n, x e r.

```
long fatr(long n) {
  if (n <= 1)              // caso base
    return 1;
  return n * fatr(n - 1);  // caso geral
}
```

pilha (topo em cima)	na ida	na volta
fatr(1)	caso base	devolve 1
fatr(2)	chama fatr(1)	2 × 1 = 2
fatr(3)	chama fatr(2)	3 × 2 = 6
fatr(4)	chama fatr(3)	4 × 6 = 24

Na prova: desenhe a pilha a cada chamada (aparece nas listas para fatorial(6) e fibonacci(5)). fat(6) = 720.

// 07 · recursão

Recursão × iteração

O que é. Recursão costuma ser mais clara e concisa; iteração costuma ser mais eficiente, pois não empilha va-

riáveis a cada chamada. O fibo recursivo recalcula os mesmos valores várias vezes.

```c
int soma(int v[], int n) {          // soma v[0..n]
  if (n == 0) return v[0];
  return v[n] + soma(v, n - 1);
}                                   // soma(v, tam-1)
long fibo(long n) {                 // 1 1 2 3 5 8
  if (n <= 2) return 1;
  return fibo(n - 1) + fibo(n - 2); // 2 chamadas
}
long fat(long n) {                  // iterativo
  long r = 1;
  for (int i = 1; i <= n; i++) r = r * i;
  return r;
}
```

`por_que_usar()`	Escolher conscientemente: clareza do código × eficiência do algoritmo.
`quando_usar()`	Recursão quando deixa o código bem mais simples (árvores de chamadas, Hanoi, mergesort).
`quando_NAO_usar()`	Recursão para fatorial, soma e Fibonacci simples: o laço é mais rápido e econômico.

`// 07 · recursão`

Potência rápida

O que é. A versão simples faz n chamadas: $x^n = x \cdot x^{n-1}$. A rápida usa n/2: n par → $(x^{(n/2)})^2$; n ímpar → $x \cdot (x^{((n-1)/2)})^2$. No máximo $\lceil \log_2 n \rceil + 1$ chamadas.

```c
long pot(long x, long n) {
  long aux;
  if (n == 0) return 1;               // base
  if (n % 2 == 0) {                   // n par
    aux = pot(x, n / 2);
    return aux * aux;
  }
  aux = pot(x, (n - 1) / 2);          // n ímpar
  return x * aux * aux;
}
// pot(2, 10): n = 10 → 5 → 2 → 1 → 0
```

`por_que_usar()`	Corta o problema pela metade a cada chamada: muito mais rápido que n voltas.
`quando_usar()`	Potências grandes; exemplo clássico de recursão MAIS eficiente que o laço.
`quando_NAO_usar()`	A versão x · pot(x, n − 1): n chamadas + custo da pilha; aí o for é melhor.

`// 07 · recursão`

Mergesort: divisão e conquista

O que é. Divisão: quebra v[l..r] ao meio em dois subvetores. Conquista: ordena cada metade recursivamente e intercala (merge) as duas já ordenadas. Caso base: tamanho 0 ou 1.

```c
void mergesort(int *v, int l, int r) {
  int m = (l + r) / 2;
  if (l < r) {                // mais de 1 elemento
    mergesort(v, l, m);       // divisão
    mergesort(v, m + 1, r);
    merge(v, l, m, r);        // conquista:
  }                           // intercala metades
}
mergesort(v, 0, tam - 1);
```

`por_que_usar()`	Problemas que se quebram em partes independentes ficam simples de resolver.
`quando_usar()`	Ordenar vetores grandes; problemas resolvidos juntando soluções de metades.
`quando_NAO_usar()`	Vetores pequenos, onde o custo das chamadas recursivas não compensa.

`// 07 · recursão`

Torres de Hanoi

O que é. Mover n discos de A para C, um por vez, nunca um maior sobre um menor. Por indução: move n−1 de A→B (C ajuda), move o maior A→C, move n−1 de B→C (A ajuda).

```c
void hanoi(int n, char ini, char fim, char aux) {
  if (n == 1)
    printf("Mova disco 1 de %c para %c\n", ini, fim);
  else {
    hanoi(n - 1, ini, aux, fim);  // n-1: A → B
    printf("Mova disco %d de %c para %c\n",
           n, ini, fim);          // maior: A → C
    hanoi(n - 1, aux, fim, ini);  // n-1: B → C
  }
}
hanoi(4, 'A', 'C', 'B');
```

`por_que_usar()`	A prova por indução já entrega o algoritmo e a garantia de que ele funciona.
`quando_usar()`	Problemas definidos por versões menores de si mesmos.
`quando_NAO_usar()`	Procurar um laço simples aqui: a solução natural é a recursiva ($2^n - 1$ movimentos).

`// 07 · recursão`

Recursão e backtracking

O que é. Enumera todas as possibilidades. Ex.: soluções inteiras de x1 + ... + xn = C. Sem saber n não dá para escrever n − 1 for aninhados: cada chamada fixa xk e chama para k + 1.

```c
void solution(int n, int C, int k, int R, int x[]) {
  if (k == n - 1) {               // última variável
    for (int i = 0; i <= n - 2; i++)
      printf("%d + ", x[i]);
    printf("%d = %d\n", R, C);    // x[n-1] = R
    return;
  }
  for (x[k] = 0; x[k] <= R; x[k]++)
    solution(n, C, k + 1, R - x[k], x);
}
solution(n, C, 0, C, x);   // R = o que falta
```

`por_que_usar()`	Gera todas as combinações quando o número de “laços” só é conhecido na execução.
`quando_usar()`	Permutações de uma string, combinações, busca de soluções por tentativa.
`quando_NAO_usar()`	Quando existe fórmula direta ou há possibilidades demais para enumerar.

// 07 · cai na prova

Cai na prova: Rastreando recursão

Aula 11 · imprime(vet, 0, 9) com vet = {1..10}

```c
void imprime(int v[], int i, int n) {
  if (i == n)
    printf("%d, ", v[i]);
  else {
    imprime(v, i + 1, n);
    printf("%d, ", v[i]);  // na volta
  }
}
```

saída →

10, 9, 8, 7, 6, 5, 4, 3, 2, 1, — o printf vem DEPOIS da chamada, então imprime na volta.

Lista 8 · questão 8: ff(7, 0)

```c
int ff(int n, int ind) {
  // imprime ind espaços e "ff(n, ind)"
  if (n == 1) return 1;
  if (n % 2 == 0) return ff(n / 2, ind + 1);
  return ff((n - 1) / 2, ind + 1)
       + ff((n + 1) / 2, ind + 1);
}
```

saída →

ff(7,0) → ff(3,1) → ff(1,2), ff(2,2) → ff(1,3); ff(4,1) → ff(2,2) → ff(1,3). Retorna 3.

// 07 · cai na prova

Cai na prova: Definições recursivas das listas

Lista 8 · questão 7: o que f calcula?

```c
int f(int n) {
  if (n == 0) return 0;
  return n + f(n - 1);
}
```

saída →

A soma 1 + 2 + ... + n = n(n+1)/2. Ex.: f(4) = 4 + 3 + 2 + 1 + 0 = 10.

Listas 5 e 8: mdc recursivo

```c
int mdc(int m, int n) {
  if (n == 0) return m;
  return mdc(n, m % n);
}
```

saída →

mdc(48, 18) → mdc(18, 12) → mdc(12, 6) → mdc(6, 0) = 6. fibo(5) = 5, com 9 chamadas.

// regra de ouro

Todo malloc tem seu free. Todo fopen tem seu fclose. Toda recursão tem seu caso base.

// 08 · checklist

Pegadinhas para revisar antes da prova

- else pertence ao if mais próximo; = atribui, == compara
- do-while executa ao menos 1 vez; while e for podem não executar
- return encerra a função na hora
- Variável simples vai por cópia; vetor vai por endereço
- Local esconde global de mesmo nome; parâmetro é local
- Matriz em função: só a 1ª dimensão pode ser omitida
- struct: b = a copia tudo; leitura é campo a campo
- Selection até tam−2 · bubble encurta · insertion empurra
- Binária só com vetor ordenado: $\log_2 n$ acessos
- & endereço · * conteúdo · NULL antes de usar
- Todo malloc tem free; nunca use memória após o free
- fopen pode dar NULL; w apaga; leia até EOF; fclose
- fread devolve 0 no fim; fseek com SEEK_SET/CUR/END
- Recursão: caso base + chamada com caso menor